

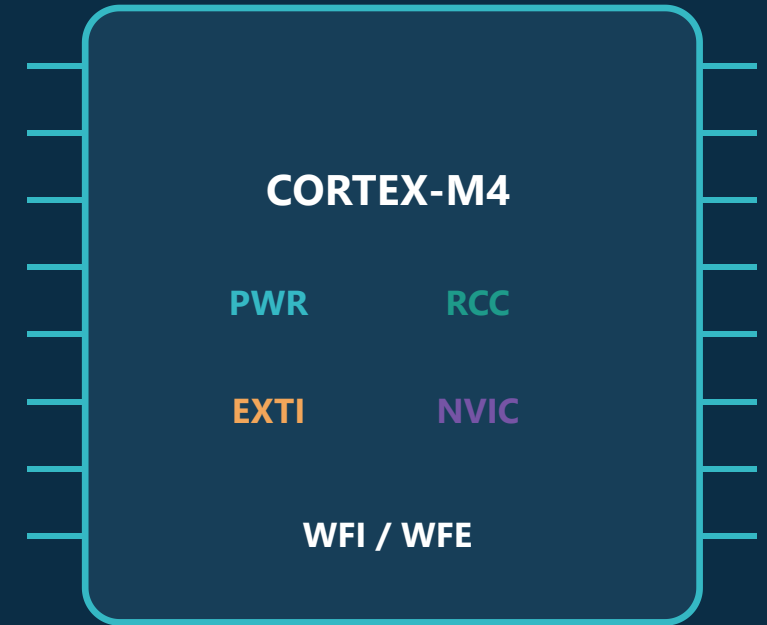


INSA SUMMER CAMP

STM32L476RG

Name: Haimanot Desie
ID: CTC-2130-2

TASK 2



STM32L476RG

KEY DEVICE FACTS

80 MHz

Cortex-M4 + FPU

Up to 1 MB Flash

Up to 128 KB SRAM

Ultra-low-power family

Shutdown supported

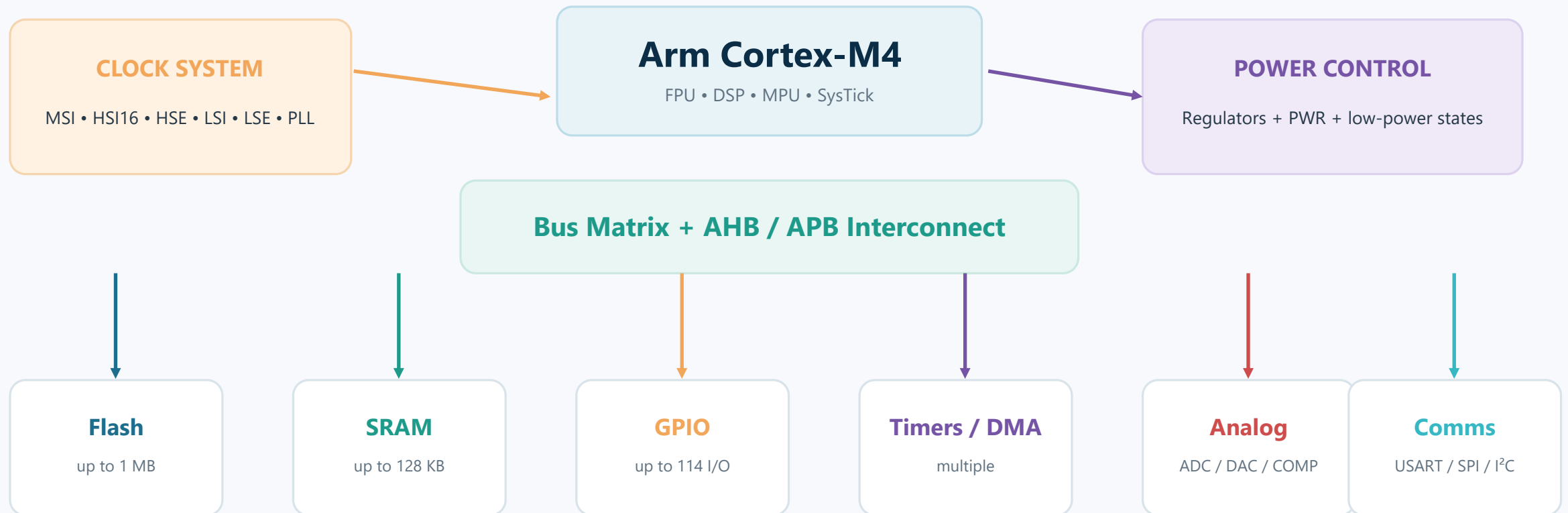
Stop 0/1/2

Standby

- Cortex-M4 core supports WFI/WFE and deep-sleep control.
- RCC + PWR provide fine control of clocks, regulators, and low-power states.
- EXTI + NVIC provide a clear interrupt/wake-up architecture.
- The family includes Sleep, Low-power Sleep, Stop 0/1/2, Standby, and Shutdown.
- RTC and low-power timers support time-based wake-up use cases.

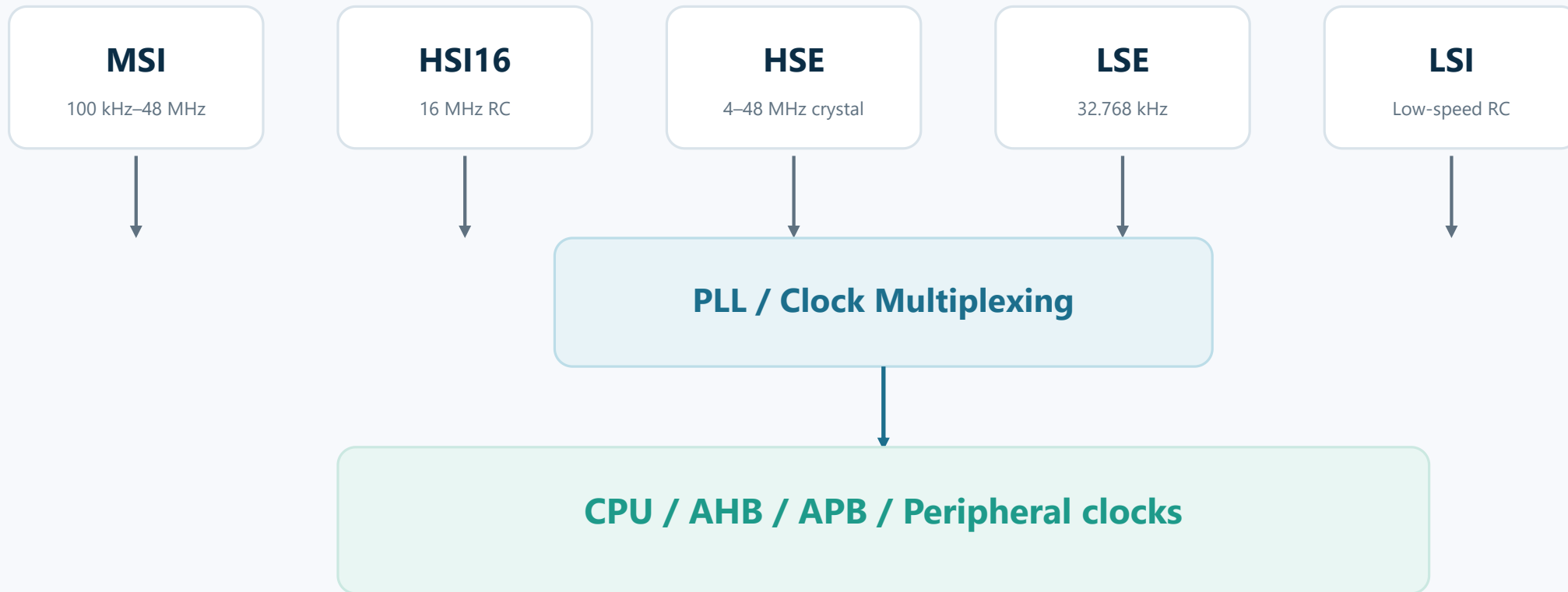
1. STM32 Architecture

High-level view of the STM32L476RG architecture.



Clock System (RCC)

RCC selects clock sources, distributes clocks, and disables them to save power.



Low-power idea: stop unnecessary clock sources and peripheral clocks before entering deeper modes.

GPIO, EXTI and NVIC

A typical external interrupt path from a GPIO pin to the Cortex-M4.



Important GPIO registers

MODER • OTYPER • OSPEEDR • PUPDR • IDR • ODR • BSRR • AFR

Low-power relevance: configure unused pins to safe states and disable unnecessary analog/digital activity.

Power-Control System (PWR)

PWR determines regulator behavior and selects the deeper low-power state.

PWR REGISTERS

PWR_CR1	LPMS • LPR • VOS
PWR_CR2	Voltage / USB-related control
PWR_CR3	Wake-up / supply control
PWR_CR4	SRAM / wake / pull controls
PWR_SR1/2	Wake / status flags
PWR_SCR	Clear wake flags

POWER PATH

Main Regulator



Low-Power Regulator



LPMS + SLEEPDEEP → deeper modes

deeper modes progressively shut down clocks and power domains while retaining only the state/resources needed for wake-up.

2. STM32L476RG Low-Power Modes

Mode	CPU / clock effect	Regulator state	Typical wake-up	Use case
Sleep	CPU clock OFF; other clocks unchanged	MR + LPR ON	Interrupt / event	Wait while peripherals stay active
Low-power Sleep	CPU clock OFF	MR OFF; LPR ON	Interrupt / event	Lower power than Sleep
Stop 0	Deep-sleep; clocks largely OFF	MR + LPR ON	EXTI / peripheral event	Low power + fast restart
Stop 1 / 2	Deeper clock shutdown	LPR based	EXTI / supported peripherals	Battery / sensor applications
Standby	Core power domain OFF	MR + LPR OFF	WKUP pin / RTC etc.	Very low-power standby
Shutdown	Deepest mode	MR + LPR OFF	Limited wake-up sources	Maximum battery life

Sleep vs Low-power Sleep

Both stop the CPU, but the regulator configuration differs.

SLEEP

SLEEP DEEP = 0

- Enter using WFI or WFE.
- CPU clock turns OFF; other clocks are not automatically stopped.
- Main and low-power regulators remain enabled.
- Wake-up by an interrupt or event.

LOW-POWER SLEEP

SLEEP DEEP = 0

- Enter from Low-power Run using WFI/WFE.
- CPU clock turns OFF.
- Main regulator OFF; low-power regulator remains ON.
- Wake-up by interrupt or event.

Key difference: Low-power Sleep uses the low-power regulator path.

Stop 0, Stop 1 and Stop 2

Deep-sleep modes with increasing power reduction and additional restrictions.

Stop 0

LPMS=000 + SLEEPDEEP

Regulators MR ON; LPR ON

Clocks All clocks OFF except LSI/LSE

Wake-up EXTI + specific peripheral events

Stop 1

LPMS=001 + SLEEPDEEP

Regulators MR OFF; LPR ON

Clocks Similar deep clock shutdown

Wake-up EXTI + supported wake sources

Stop 2

LPMS=010 + SLEEPDEEP

Regulators LPR used

Clocks Very low-power clock state

Wake-up EXTI + selected wake-capable peripherals

Standby and Shutdown

The deepest states remove the core power domain for very low current consumption.

STANDBY

~120 nA typical (specified conditions)

- PWR regulators OFF.
- Core state is not retained as in Stop; selected backup resources can remain.
- Wake-up via WKUP pins, RTC events and other supported sources.
- Use when long battery life matters more than instant execution continuity.

SHUTDOWN

~30 nA typical (specified conditions)

- Both main and low-power regulators OFF.
- Almost all internal functions powered down.
- Wake-up is limited to supported sources such as WKUP pins and RTC.
- Maximum power saving; treat wake-up as a restart-like path.

3. Registers (PWR and RCC)

The assignment requires register-level understanding, not only mode names.

PWR

PWR_CR1	LPMS[2:0]	Selects low-power mode
PWR_CR1	LPR	Selects low-power regulator path
PWR_CR3	Wake-up / supply	Controls wake/supply behavior
PWR_SR1	WUFx / SBF	Reports wake / standby status
PWR_SCR	CWUFx / CSBF	Clears wake / standby flags

RCC

RCC_CR	Oscillator enable / readiness
RCC_CFGR	Clock selection / STOPWUCK
RCC_PLLCFGR	PLL configuration
RCC_AHBxENR / APBxENR	Peripheral clock enable
RCC_BDCR / CSR	Low-speed / backup-domain clocks

Cortex-M4 System Control Register (SCR)

The Cortex-M core uses SCR to distinguish ordinary Sleep from Deep Sleep.

SEVONPEND

bit 4

SLEEPDEEP

bit 2

SLEEPONEXIT

bit 1

SLEEPDEEP = 0

WFI / WFE enters ordinary Sleep.

Use when the CPU should sleep but you do not need a deep-stop mode.

SLEEPDEEP = 1

WFI / WFE can enter Stop / Standby / Shutdown after PWR configuration.

PWR + SCR must agree: PWR selects the mode; SCR requests deep sleep.

EXTI and NVIC Registers

EXTI creates the interrupt/event condition; NVIC delivers the interrupt to the Cortex-M4.

EXTI (selected registers)

EXTI_IMR1 Interrupt mask

EXTI_EMR1 Event mask

EXTI_RTSTR1 Rising trigger

EXTI_FTSR1 Falling trigger

EXTI_PR1 Pending status

EXTI_SWIER1 Software trigger

Typical path: **GPIO → EXTI → NVIC → ISR**

NVIC

ISER Interrupt Set-Enable

ICER Interrupt Clear-Enable

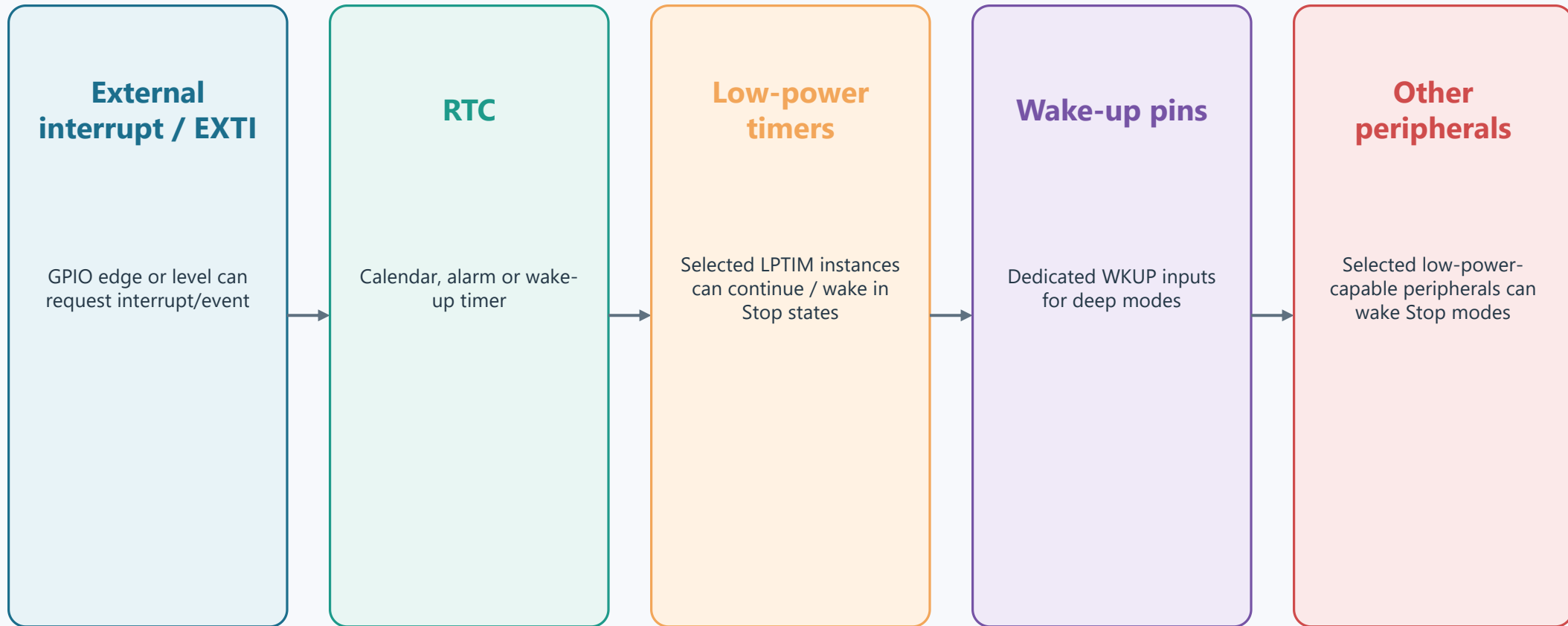
ISPR Set Pending

ICPR Clear Pending

IPR Priority

4. Wake-Up Sources

Wake-up capability depends on the selected low-power mode.



Design rule: always verify the wake-up source in the RM0351 mode table before choosing a low-power mode.

5. WFI and WFE

Two Cortex-M4 sleep instructions with different wake-up semantics.

WFI

Wait For Interrupt

- CPU suspends until an interrupt/exception wakes it.
- Typical for interrupt-driven firmware.
- Common low-power sequence: configure wake source → WFI → ISR.

WFE

Wait For Event

- CPU suspends when the event register is 0.
- Events can come from SEV, peripherals, and pending interrupts when SEVONPEND is set.
- Useful for event-driven synchronization and event wake-up schemes.

WFI vs WFE — Practical Example

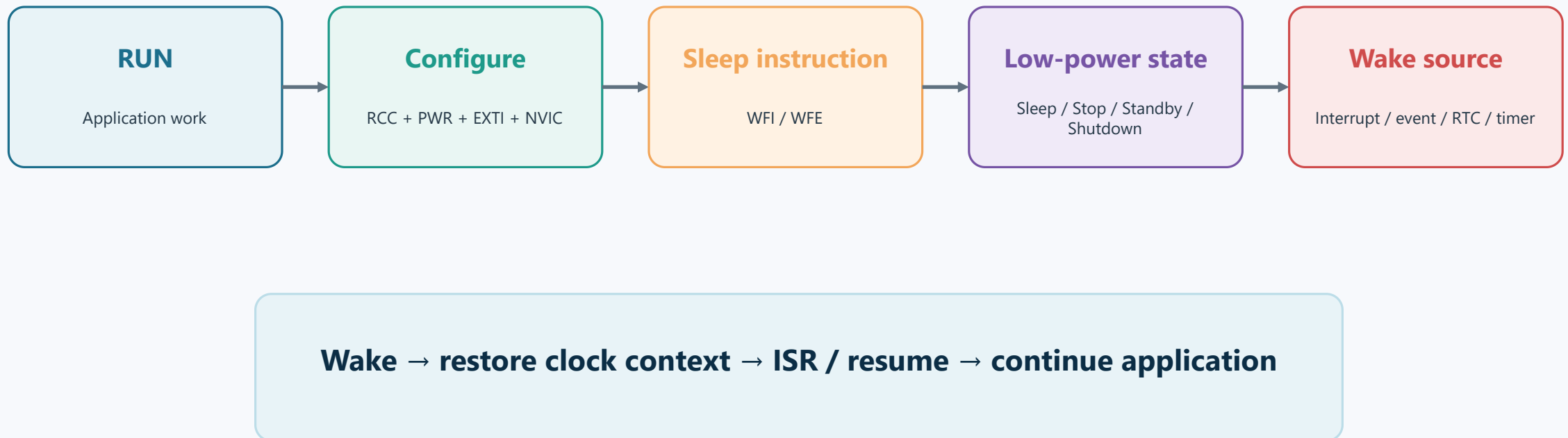
Choose based on whether the design is interrupt-driven or event-driven.

Aspect	WFI	WFE
Primary concept	Interrupt	Event
Hardware path	Peripheral / EXTI → NVIC	Event source / SEV / SEVONPEND
Best fit	Sensor interrupt, button, UART receive	Event synchronization, event-driven coordination
Typical code	<code>__WFI();</code>	<code>__WFE();</code>

For low-power embedded control: “wait for my interrupt” → WFI; “wait for an event” → WFE.

Overall STM32 Low-Power Sequence

How the pieces fit together in firmware.



Deepest modes use the most restrictive wake-up sources; verify every mode in RM0351 before implementation.

Conclusion

1

STM32L476RG

Cortex-M4, 80 MHz, ultra-low-power architecture.

2

PWR + RCC

PWR selects low-power behavior; RCC controls clock sources and enables.

3

SCR + WFI/WFE

SLEEPDEEP chooses ordinary Sleep vs Deep Sleep; WFI is interrupt-oriented, WFE is event-oriented.

4

EXTI + NVIC

Build the interrupt/wake path from GPIO/EXTI to the Cortex-M4.

5

Mode selection

Choose the deepest mode that still supports the required wake-up source.

References

- 1 **STM32L476RG Datasheet — DS10198 (Rev. 11)**
- 2 **STM32L4 Reference Manual — RM0351**
- 3 **STM32 Cortex-M4 Programming Manual — PM0214**
- 4 **STM32L476RG Product Page**